

Image Processing and Machine Vision

Assignment 02: Fitting and Alignment

October, 2025

[GitHub Repository](#)

220197E Gunathilaka K. L.

Q1. LoG blob detection on sunflowers

Sunflower centers were detected using a Laplacian-of-Gaussian (LoG) scale space with non-maximum suppression across scale and space. A range $\sigma \in [1, 15]$ (25 scales) was used, with a response threshold of 1.3 and an overlap suppression of 0.5. Estimated radius were mapped by $r = \sqrt{2} \sigma$.

```

1 THRESH_ABS = 0.27           # LoG response threshold
2 SPATIAL_NEIGHBOR = 3        # 2D neighborhood for local maxima
3 SCALE_NEIGHBOR = 3          # Number of scales to check above/below
4 responses = []
5 for idx, sigma in enumerate(sigmas, 1):
6     blur = gaussian_filter(gray, sigma=sigma)
7     lap = laplace(blur)
8     log_resp = (sigma**2) * np.abs(lap)
9     responses.append(log_resp)

```

Listing 1: LoG scale space with scale-normalized Laplacian.

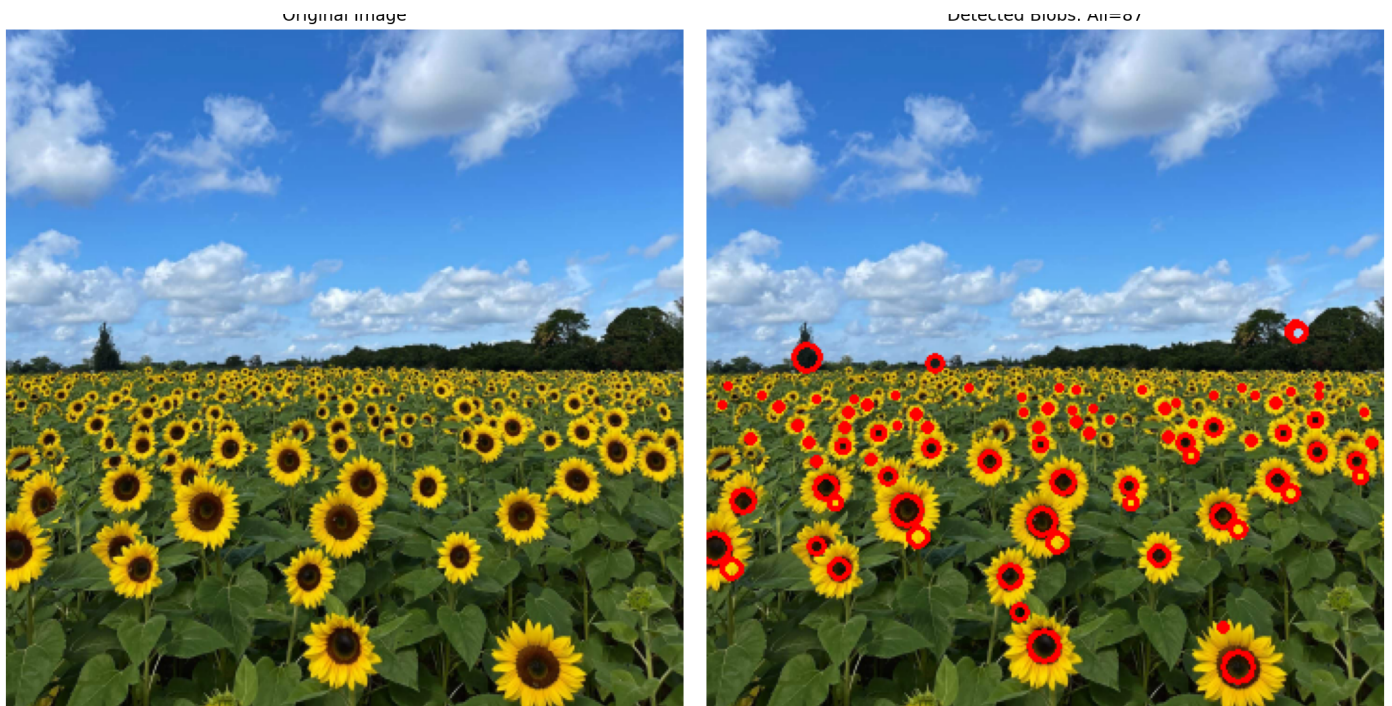


Figure 1: Original image (left) and detected blob centers (right).

Top 5 largest blobs: (1) $C = (106, 255)$, $r = 8.49$, $\sigma = 6.00$, response 0.363; (2) $C = (4, 275)$, $r = 8.49$, $\sigma = 6.00$, 0.288; (3) $C = (179, 327)$, $r = 8.49$, $\sigma = 6.00$, 0.345; (4) $C = (282, 338)$, $r = 8.49$, $\sigma = 6.00$, 0.308; (5) $C = (53, 174)$, $r = 7.74$, $\sigma = 5.47$, 0.301.

Q2. RANSAC fitting

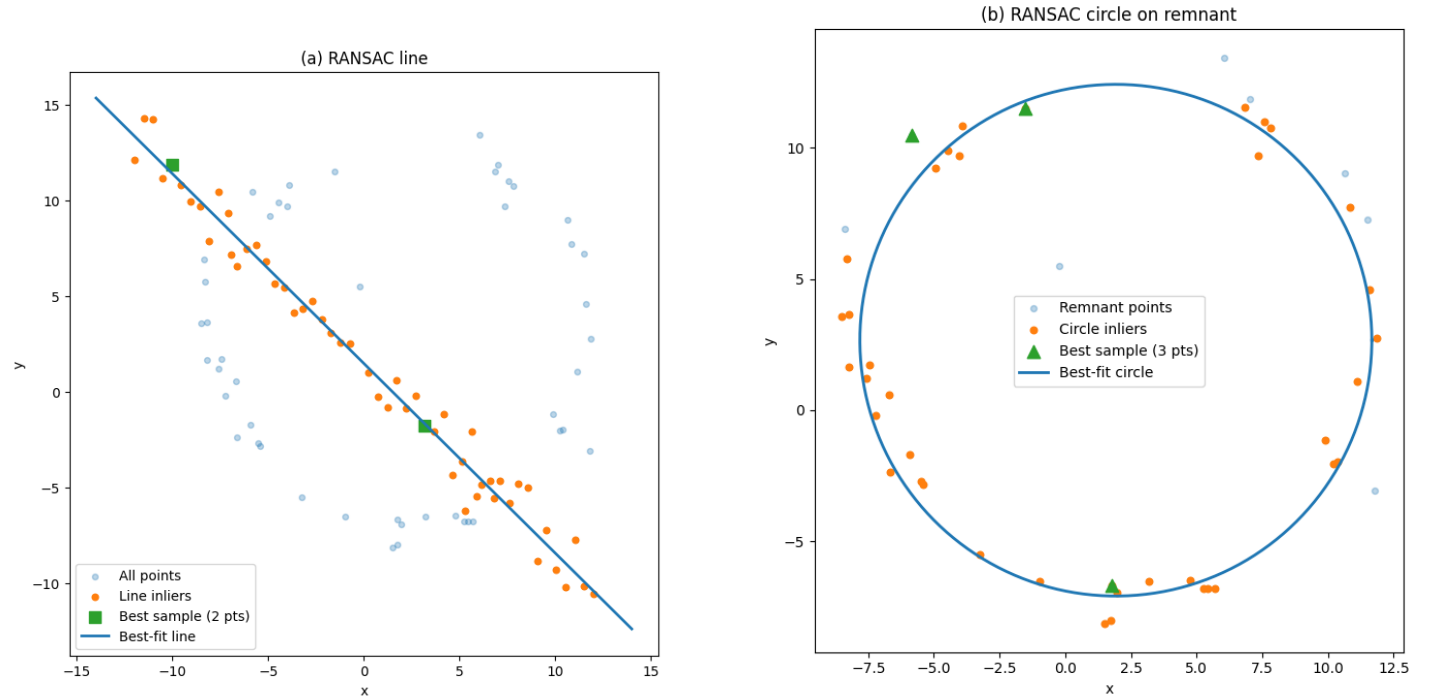
A line was fitted by RANSAC with the normalized implicit model $ax + by + d = 0$ ($\| [a, b] \| = 1$). Inliers were selected by normal distance (< 0.5). After removing these points, a circle was fitted to the remnant via RANSAC using radial error (< 0.5).

Why not fit the circle first? As many line points sit near some radius R and pass the radial-error test, and near-collinear samples yield huge-radius circles that falsely gain high consensus. The line model has a stable distance and larger true support, so fitting the line first removes the dominant structure and makes the subsequent circle fit reliable.

RANSAC line (with TLS refine). A robust line was estimated by repeating random two-point samples (`iters` = 1000). For each sample, the direction $v = q - p$ and unit normal $(a, b) = (v_y, -v_x) / \|v\|$ defined the implicit line $ax + by + d = 0$ with $d = -(ap_x + bp_y)$. Point-line distances $|ax_i + by_i + d|$ were computed; points within `thr` = 1.6 were counted as inliers. The hypothesis with the largest consensus (at least `min_consensus` = 20) was then refined by total least squares: inliers were mean-centered and an SVD provided the unit normal (last right-singular vector), with d reset using the centroid.

RANSAC circle (on line-free remnant, with algebraic refine)- After removing line inliers, random triples from X_{rem} were used to form a candidate circle via 3×3 determinant formulas; degenerate (nearly collinear) triples ($\det A \approx 0$) and implausible radii ($R \geq R_{\text{cap}} = 100$) were rejected. For each candidate (x_0, y_0, R) , radial errors $|\| [x_i, y_i] - [x_0, y_0] \| - R|$ were scored with `thr_c` = 1.5 and a minimum consensus of 10. The best circle was refined by solving $[x \ y \ 1] [c_1, c_2, c_3]^T = x^2 + y^2$ on inliers, yielding $x_0 = c_1/2$, $y_0 = c_2/2$, and $R = \sqrt{c_3 + x_0^2 + y_0^2}$.

Note: Explanation of code is added as the code is too long to be included.



(a) RANSAC line: best sample, inliers, and TLS fit.

(b) RANSAC circle on the line-subtracted remnant.

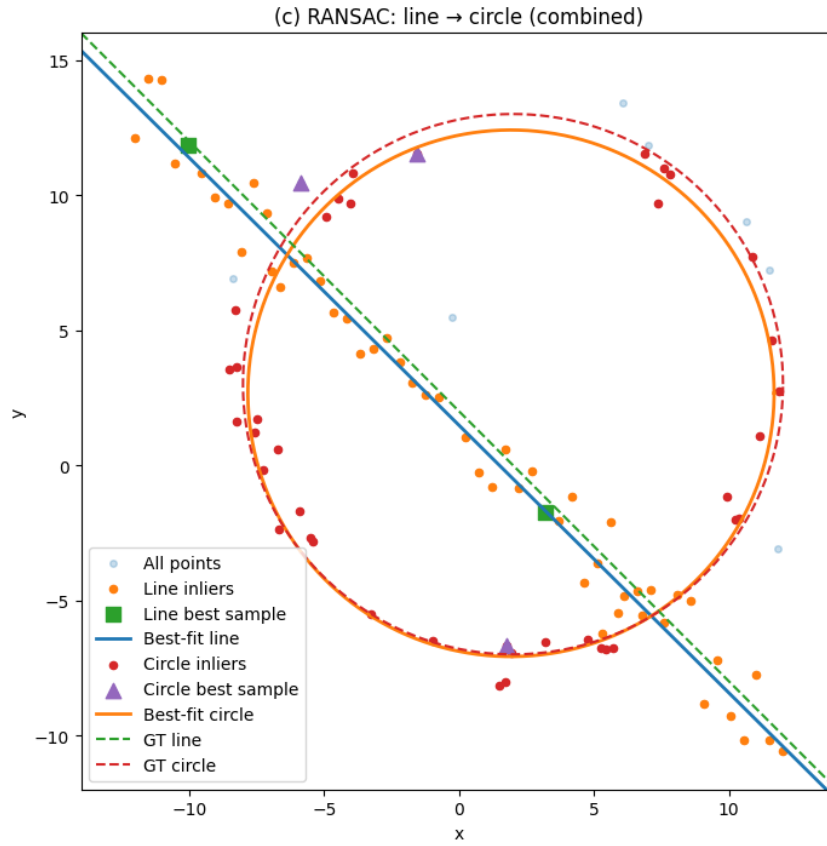


Figure 3: Combined result: final line and circle fits.

Q3. Planar homography

A planar homography was estimated from four point correspondences using the Direct Linear Transform (DLT) solved by SVD and normalized to $H_{33} = 1$. Backward warping with bilinear sampling was applied to project the source onto the destination plane.

```

1 dst_pts = np.asarray(dst_pts, dtype=np.float32)
2 sh, sw = src.shape[:2]
3 src_pts = np.float32([[0, 0], [sw - 1, 0], [sw - 1, sh - 1], [0, sh - 1]])
4 H = cv.getPerspectiveTransform(src_pts, dst_pts) # Compute homography and warp
5 warped = cv.warpPerspective(src, H, (dst.shape[1], dst.shape[0]))
6 mask = np.zeros(dst.shape[:2], np.uint8) # Create mask and blur edges
7 cv.fillConvexPoly(mask, dst_pts.astype(np.int32), 255)
8 mask_f = (mask.astype(np.float32) / 255.0)[..., None] # Alpha blend
9 out = (dst * (1 - mask_f) + warped * mask_f).astype(np.uint8)

```

Listing 2: DLT homography and warping.



Figure 4: Source image (left), target plane (center), and warped result (right).

The blank billboard provides a near-ideal plane, producing a perspective-consistent placement of the poster.

Q4. SIFT matching + RANSAC homography + stitching

Keypoints and descriptors were obtained with SIFT, then matched using BFMatcher (KNN, $k = 2$) with Lowe's ratio test (0.75). A homography was estimated via custom RANSAC+DLT using reprojection error (< 3 px). The final panorama was created by translating the canvas to positive coordinates, warping the source with the estimated H , and blending with simple masking.

```
1 sift = cv.SIFT_create()
2 kp1, des1 = sift.detectAndCompute(img1_gray, None)
3 kp5, des5 = sift.detectAndCompute(img5_gray, None)
4
5 matches = cv.BFMatcher().knnMatch(des1, des5, k=2)
6 good = [m for m,n in matches if m.distance < 0.75*n.distance]
7 ...
8 pts1 = np.float32([kp1[m.queryIdx].pt for m in good])
9 pts5 = np.float32([kp5[m.trainIdx].pt for m in good])
10 ...
11 H_custom, inliers_idx = find_homography_RANSAC(pts1, pts5, reproj_thresh=3.0) #my
    custom function
12 ...
13 stitched_custom = stitch_images(img1, img2, H_custom)
```

Listing 3: SIFT + ratio test + RANSAC homography.



Figure 5: Top: input images. Bottom-left: SIFT matches. Bottom-right: stitched panorama.